

Claude Code — Learner Prompt Packet

Every prompt taught in the course, copy-paste ready. Day 1 prompts were re-verified against the Day 1 deck and labs on 2026-08-10; Day 2 prompts were rebuilt and re-verified against the corrected Day 2 labs on 2026-08-14. The Day 1 review workflow below uses the current non-colliding `/dashboard-review` name taught by the Day 2 migration.

Day 1 — Fundamentals

The Agentic Loop & Setup — Paste this: first contact

Use it for: What you are practicing

```
claude
```

```
/doctor
```

```
Create hello.md with one line explaining what Claude Code is, then tell me what you did.
```

Memory & CLAUDE.md — Paste this: memory workflow

Use it for: What you are practicing

```
/init
```

```
Remember: in this project we use double quotes and 2-space indents. Save that so you never suggest otherwise.
```

```
Write HANDOFF.md: what you tried, what worked, what failed, what is next - so a fresh session can continue from that file alone.
```

Context Mastery — Paste this: sharp delegation

Use it for: What you are practicing

In `@server.js`, add input validation to the `POST /api/metrics` route, following the validation already on `POST /api/tasks`. Then run the server and prove it rejects a bad request.

In `@server.js`, make `GET /api/tasks` return newest first. Change only that one query, and show me the diff.

Commands, Cost & Permissions — Build a human-invoked skill

Use it for: What you are practicing

Create `.claude/skills/dashboard-review/SKILL.md` with name `dashboard-review`, description `'Review a target for security, performance, and style'`, `disable-model-invocation: true`, `allowed-tools Read, Grep, Glob, Bash(git diff --name-only)`, `disallowed-tools Write, Edit`; body: `review $ARGUMENTS` and return prioritized findings with file references.

```
/dashboard-review server.js
```

Day 2 — Extensions

Commands follow-along — Build a human-invoked `/test-this` skill

Create `.claude/skills/test-this/SKILL.md`: name `test-this`; `disable-model-invocation: true`; `allowed-tools: Read, Write, Edit, Bash`. Body: `write and run tests for $ARGUMENTS`, covering happy path, edges, and errors.

```
/test-this src/utils/helpers.js
```

Lab 2A — Build a model-invoked skill that triggers reliably

Create `.claude/skills/dashboard-code-review/SKILL.md` with name `dashboard-code-review`; description: `Reviews code for security, quality, and style issues. Use when the user says "review this code", "check this file", "code review", "audit this", "look for issues", "quality check", "security check", "find bugs", or "QA check". Do NOT use for writing new code, refactoring, or documentation; disable-model-invocation false; allowed-tools Read, Grep, Glob; disallowed-tools Write, Edit, Bash. Review hardcoded secrets, SQL injection, unsanitized input, functions over 50 lines, missing error handling, unused imports, naming conventions, formatting consistency, and the three project-specific checks recorded in Lab 2A Exercise 1. Return [RED] Critical (security), [YELLOW] Warning (quality), and [BLUE] Info (style). If no issues are found, explicitly state "No issues found" and do not invent problems.`

Run the five activation prompts from Lab 2A with a clean context between runs. Record whether dashboard-code-review loaded, whether the RED/YELLOW/BUE contract appeared, and whether the negative implementation prompt correctly did not trigger it.

Lab 2B — Build and harden hooks, then add `/dashboard-review`

Create the PostToolUse Write|Edit formatter hook from Lab 2B. Use the pinned local Prettier dependency or instructor no-network fallback; do not download a package during class. Merge the hook into `.claude/settings.json` without overwriting permissions.

Create the PreToolUse secret detector from Lab 2B. Advisory stage: on the exact fixture `const API_KEY = "FAKE_TEST_SECRET_12345"`, return valid stdout JSON with `systemMessage "WARNING: possible hardcoded secret"` and `exit 0`. Blocking stage: return an actionable stderr reason and `exit 2`. Then widen the matcher from Write to Write|Edit and inspect both `tool_input.content` and `tool_input.new_string`. Prove the unsafe Write and Edit cases are blocked and `process.env.API_KEY` is allowed.

Create `.claude/skills/dashboard-review/SKILL.md` with name `dashboard-review`, `disable-model-invocation true`, `allowed-tools Read, Grep, Glob, Bash(git diff --name-only)`, and `disallowed-tools Write, Edit`. Review `$ARGUMENTS` using the dashboard-code-review checklist and return `critical/warning/info totals`, the top three fixes, and `APPROVE, NEEDS CHANGES, or BLOCK`. If `$ARGUMENTS` is empty, use `git diff --name-only`.

Lab 2C — Evaluate variance and uplift

Create `.claude/skills/dashboard-code-review/evals/evals.json` as valid JSON with `skill_name "dashboard-code-review"` and 10 eval objects: five `should-trigger` and five `should-not-trigger`. Each object must contain `id`, `prompt`, `should_trigger`, `expected_output`, and `files`. Then create `evals/results.md` for actual activation, output-contract evidence, assertions, and pass/fail. Keep held-out wording for validation rather than tuning and grading on the same phrases.

Choose one uncertain prompt and run it five times with a clean context between runs. Report the activation band, not one point estimate. For the A/B baseline, exit the interactive session and run ``claude -p --disable-slash-commands "Review src/routes/tasks.js for issues"``; then run the identical prompt in a normal session with dashboard-code-review active. Score both against the same rubric and judge any uplift against the observed variance band.

Update the dashboard-code-review description only from observed training failures, rerun held-out validation prompts, and keep the change only if the improvement exceeds the variance band. Ensure the human-invoked `/dashboard-review` skill explicitly delegates to the same dashboard-code-review checklist so there is one source of truth.

Lab 2D — Package and ship through a local marketplace

```
Run claude plugin init dashboard-toolkit --with skills hooks. Package dashboard-code-review and dashboard-review under skills/, package the formatter and final Write|Edit secret blocker under hooks/, preserve content and new_string checks, use ${CLAUDE_PLUGIN_ROOT}-relative hook paths, remove placeholders, and validate the plugin with claude plugin validate ~/.claude/skills/dashboard-toolkit --strict.
```

```
Create team-marketplace/.claude-plugin/marketplace.json with name team-marketplace, a top-level description and owner, and dashboard-toolkit sourced from ./plugins/dashboard-toolkit. Run `claude plugin disable dashboard-toolkit@skills-dir` before installing the marketplace copy so only one plugin with that name is enabled. Add the local marketplace, install dashboard-toolkit@team-marketplace at local scope, run /reload-plugins only if the install summary requests it, invoke /dashboard-toolkit:dashboard-review server.js, and prove both unsafe Write/Edit fixtures are blocked while process.env.API_KEY remains allowed. Project-settings hooks may remain active alongside plugin hooks, so duplicate enforcement is expected until those project hooks are removed.
```

Day 3 — Orchestration & Production

MCP: connect to everything — Paste this: wire up a server

Use it for: What you are practicing

```
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

```
claude mcp add --transport http linear https://mcp.linear.app/mcp
```

```
claude mcp login linear
```

Subagents & agent teams — Paste this: nest and grade

Use it for: What you are practicing

```
Launch a research lead subagent for the payments module. Have it spawn one subagent per service, then compile a single dependency report for me.
```

```
Spawn 3 subagents to each draft the retry logic independently. Then spawn a grader subagent that scores all 3 against rubric.md and reports the winner with scores.
```

Production & CI/CD — Paste this: headless runs

Use it for: What you are practicing

```
claude -p "Review src/" --strict-mcp-config --disable-slash-commands --setting-sources user --tools "Read,Grep,Glob" --allowedTools "Read,Grep,Glob" --permission-mode dontAsk --max-turns 5
```

```
claude -p "List TODOs" --strict-mcp-config --disable-slash-commands --setting-sources user --tools "Read,Grep" --allowedTools "Read,Grep" --permission-mode dontAsk --max-turns 5 --output-format json
```

```
git diff main...HEAD | claude -p "Flag critical issues" --strict-mcp-config --disable-slash-commands --setting-sources user --tools "" --permission-mode dontAsk --max-turns 5
```

Scale, cost & safe parallel work — Paste this: parallel worktrees

Use it for: What you are practicing

```
git worktree add ../app-feature-a feature-a
```

```
git worktree add ../app-bugfix-b bugfix-b
```

```
cd ../app-feature-a && claude # session 1 - repeat in b for session 2
```